

Technology & Operations Induction Program

Hands-on Core Banking Relational Database Lab

John Michael Bilbao
EastWest Banking Corporation — Technology Induction Track, Manila Center
August 19, 2026

1. Lab Overview and Objectives

This is the write-up for the EastWest Bank core banking relational database lab. The lab asks for three things to be done to a PostgreSQL database and shown working: business rules enforced by the database itself rather than by application code, a ledger that cannot reference an account which does not exist, and a fund transfer that either happens completely or does not happen at all.

Everything below was run against PostgreSQL 16.14 on a database named `ewb_core`, twice over. The first pass runs the scripts in `sql/` through `psql`; the second performs the identical steps by hand in the pgAdmin 4 Query Tool, which is the tool the lab specifies. Both passes are reproducible: `setup.sh` builds the cluster and captures the `psql` output of every step into `results/`, and `scripts/capture_pgadmin.py` starts pgAdmin, connects it to the same cluster, executes the same SQL and screenshots the result.

No figure in this report was retyped or staged. Each `psql` figure is rendered from the capture file `setup.sh` wrote, and each pgAdmin figure is a screenshot of the running application taken immediately after Execute finished, so no figure can claim something the database did not do.

Objective	Where it is demonstrated
Enforce banking business rules with DDL constraints (PRIMARY KEY, CHECK, FOREIGN KEY)	Exercise 1, and the ledger table in Exercise 2
Maintain ledger consistency on double-entry principles	Exercise 2, and the debit/credit check in Exercise 3
Query account states, aggregates and customer ledger records	Exercise 2, Step 2.3
Execute atomic transfers with BEGIN, COMMIT and ROLLBACK	Exercise 3

2. Environment

A single-node PostgreSQL 16.14 cluster was built from scratch for the lab, holding one database, `ewb_core`. The cluster is torn down and rebuilt on every run of `setup.sh`, so no step can quietly depend on state left behind by an earlier attempt.

```
>_ Windows PowerShell - build the cluster and ewb_core

psql (PostgreSQL) 16.14

PS C:\ewb\lab> initdb -D data -E UTF8
PS C:\ewb\lab> pg_ctl -D data start
PS C:\ewb\lab> createdb ewb_core

 database | connected_as | port | version
-----+-----+-----+-----
 ewb_core | postgres     | 5432 | 16.14
(1 row)
```

Figure 1: The cluster and the ewb_core database, on PostgreSQL 16.14.

One change was made to the cluster as initdb left it. initdb writes a pg_hba.conf that trusts any TCP connection from the local machine, which means the instance can be opened without a password. For a lab whose subject is the database as the last line of defence that is the wrong default, so host connections were switched to scram-sha-256 before the server was first started. pgAdmin therefore has to authenticate, which is the first thing it does below.

Connect to Server

Please enter the password for the user 'postgres' to connect the server - "EWB Core Banking (Manila Center Lab)"

☐ Save Password

Cancel

OK

Figure 2: pgAdmin 4 refusing to open the server without the password for 'postgres'. This is pg_hba.conf doing its job, not pgAdmin being cautious.

With the password supplied, the server connects and pgAdmin shows its dashboard. The server is registered under a group named EastWest Bank so the tree reads the way the induction material does.

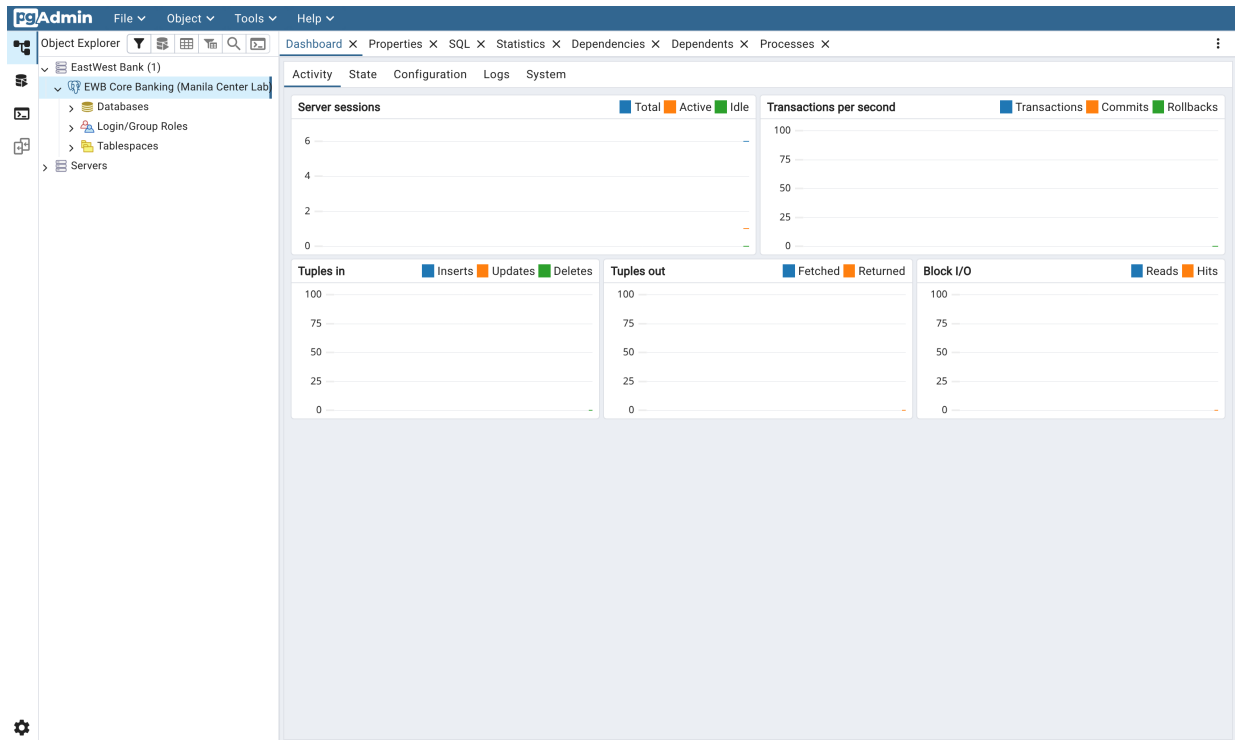


Figure 3: The EWB Core Banking instance connected in pgAdmin 4. Sessions, transactions per second and block I/O are all live counters from this cluster.

3. Exercise 1: DDL and Database-Level Safety Constraints

Application code fails. It gets deployed half-finished, it gets called by a batch job nobody remembered, and it gets bypassed entirely by an operator with psql open. A rule that only exists in application code is therefore a rule that holds only most of the time. The two rules this exercise cares about — a balance is never negative, and a currency is one EWB has approved — are written into the table definition instead, where every writer has to go through them.

Step 1.1: Creating the account master table

ewb_accounts carries five columns and four rules: a PRIMARY KEY on the account number, NOT NULL on the customer name, and the two named CHECK constraints. The CHECKs are given explicit names rather than left to PostgreSQL to generate, because the constraint name is what appears in the error message, and “check_positive_balance” tells an on-call engineer what happened where “ewb_accounts_check1” does not.

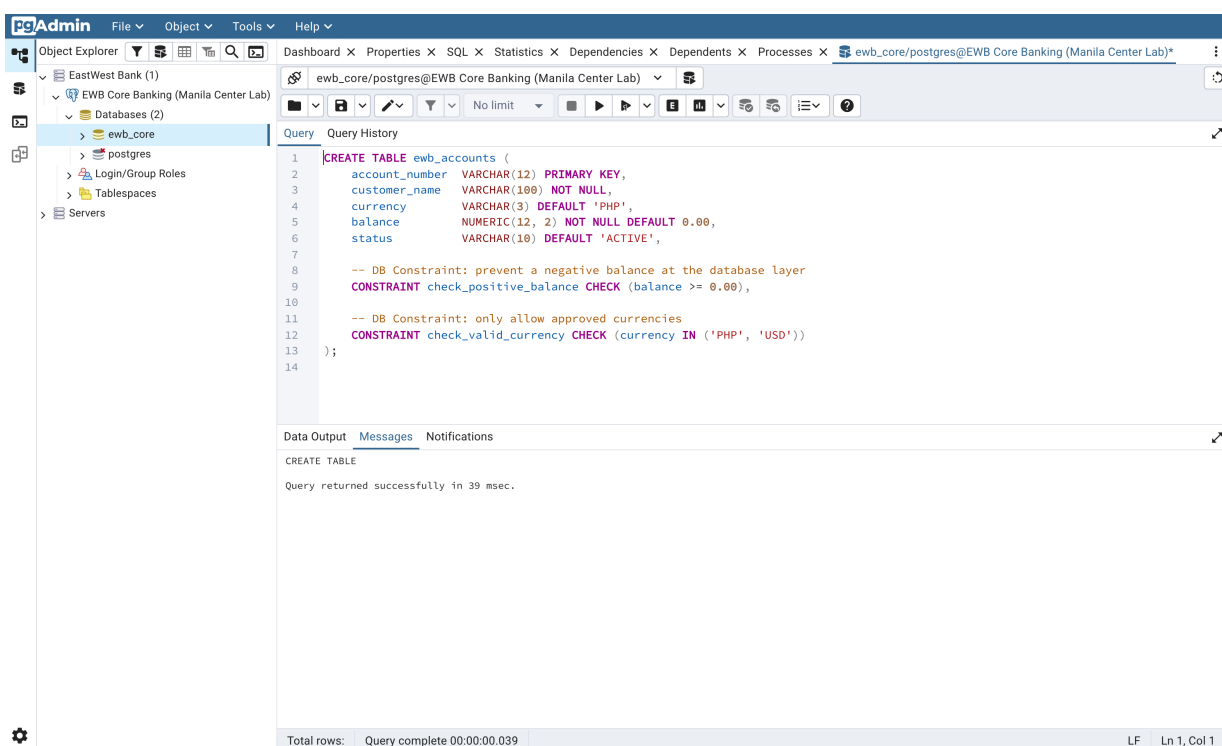


Figure 4: Step 1.1 in the pgAdmin 4 Query Tool. The Messages pane reports CREATE TABLE and the statement completed in 37 msec.

It is worth reading the table back rather than trusting that the statement did what it looked like it would. Two details only show up on the way out:

```
> _ psql - ewb_accounts, and its constraints
```

```
CREATE TABLE
--- the table as PostgreSQL stored it ---
                                Table "public.ewb_accounts"
  Column      |      Type      | Collation | Nullable |      Default
-----+-----+-----+-----+-----
account_number | character varying(12) |          | not null |
customer_name  | character varying(100) |          | not null |
currency       | character varying(3)  |          |          | 'PHP'::character varying
balance       | numeric(12,2)         |          | not null | 0.00
status        | character varying(10) |          |          | 'ACTIVE'::character varying
Indexes:
    "ewb_accounts_pkey" PRIMARY KEY, btree (account_number)
Check constraints:
    "check_positive_balance" CHECK (balance >= 0.00)
    "check_valid_currency" CHECK (currency::text = ANY (ARRAY['PHP'::character varying, 'USD'::character varying]::text[]))

--- the two CHECK constraints, read back from the catalog ---
 constraint_name | definition
-----+-----
check_positive_balance | CHECK ((balance >= 0.00))
check_valid_currency  | CHECK (((currency)::text = ANY ((ARRAY['PHP'::character varying, 'USD'::character varying]::text[])))
(2 rows)
```

Figure 5: The table as PostgreSQL stored it, and the two CHECK constraints read back out of pg_constraint.

First, DEFAULT 'PHP' and DEFAULT 'ACTIVE' are stored as 'PHP'::character varying, so an insert that omits those columns still produces a row the currency CHECK accepts. Second, the IN ('PHP', 'USD') that was written has been rewritten by the planner into = ANY (ARRAY[...]). That is the same test, stored in the form PostgreSQL evaluates; a fresher reading pg_constraint for the first time should expect it not to look like what they typed.

Step 1.2: Testing the safeguards on purpose

A constraint that has never been made to fire is an assumption. Both invalid inserts from the lab were run, and both were refused. This is the part of the exercise worth doing slowly, because the shape of the error is as instructive as the fact of it.

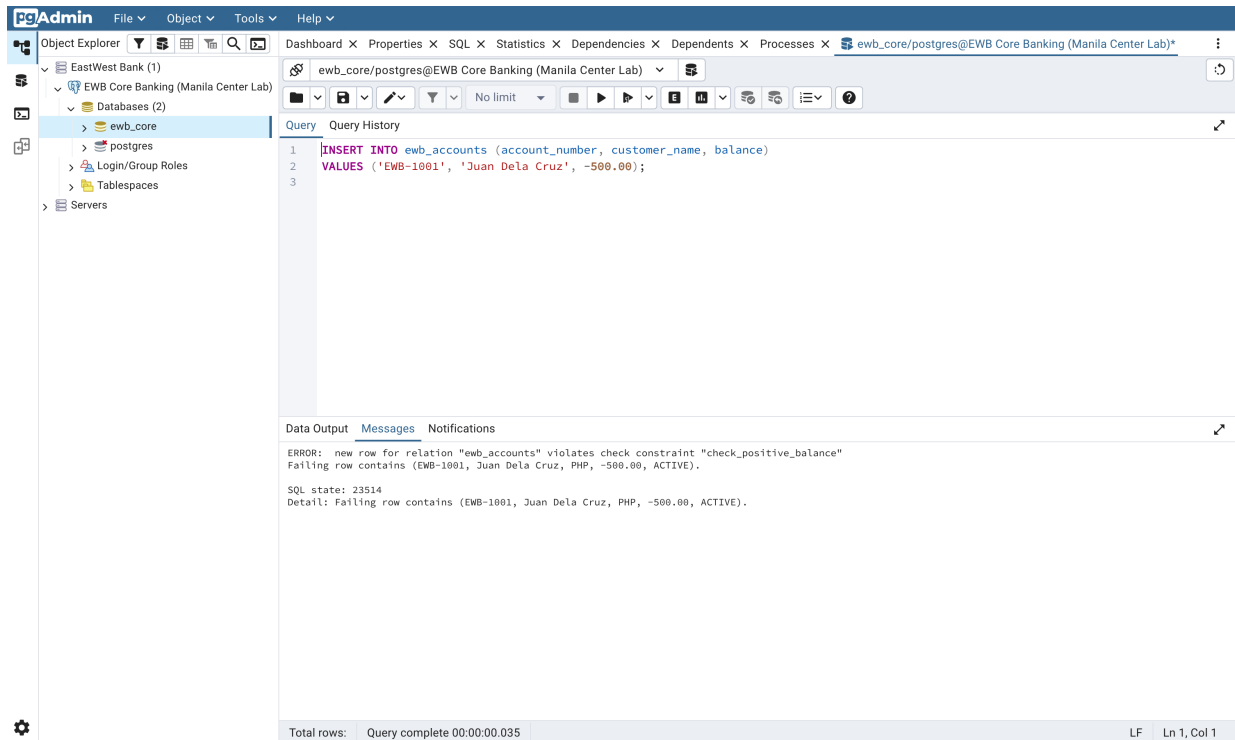


Figure 6: Attempt A: a negative opening balance. The row never reaches the table, and pgAdmin reports SQL state 23514 alongside the message.

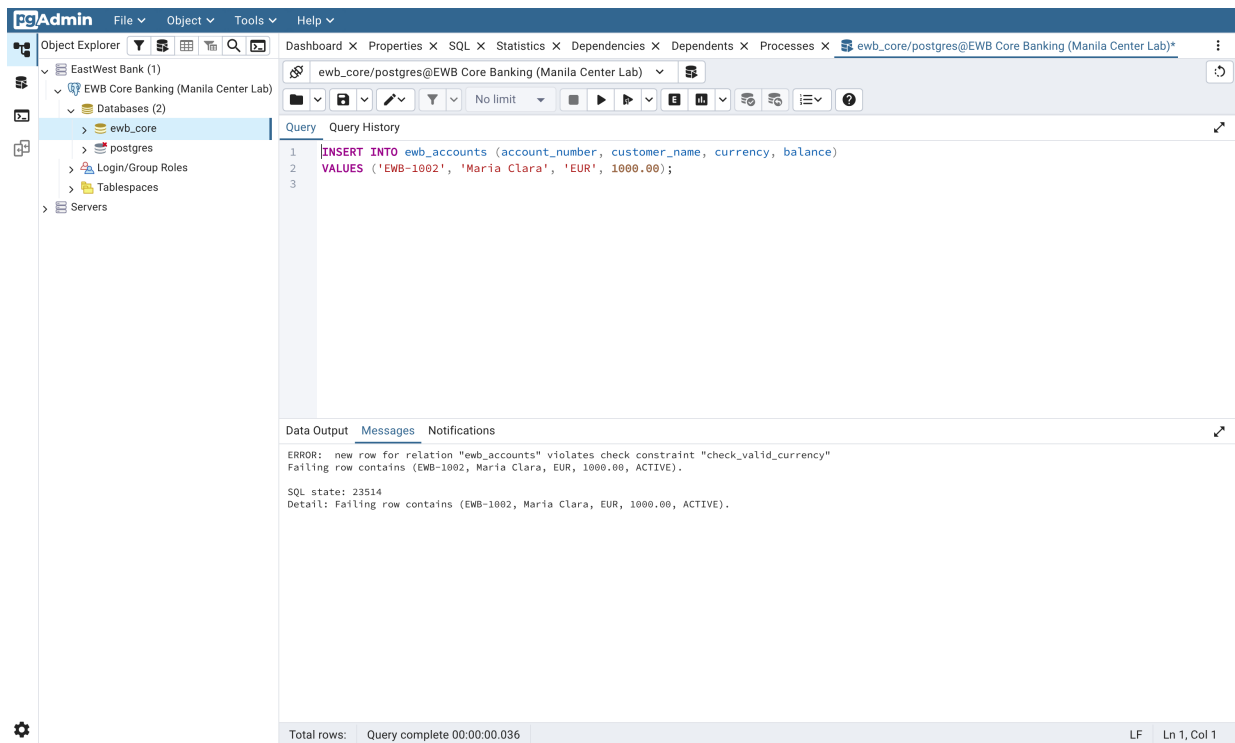
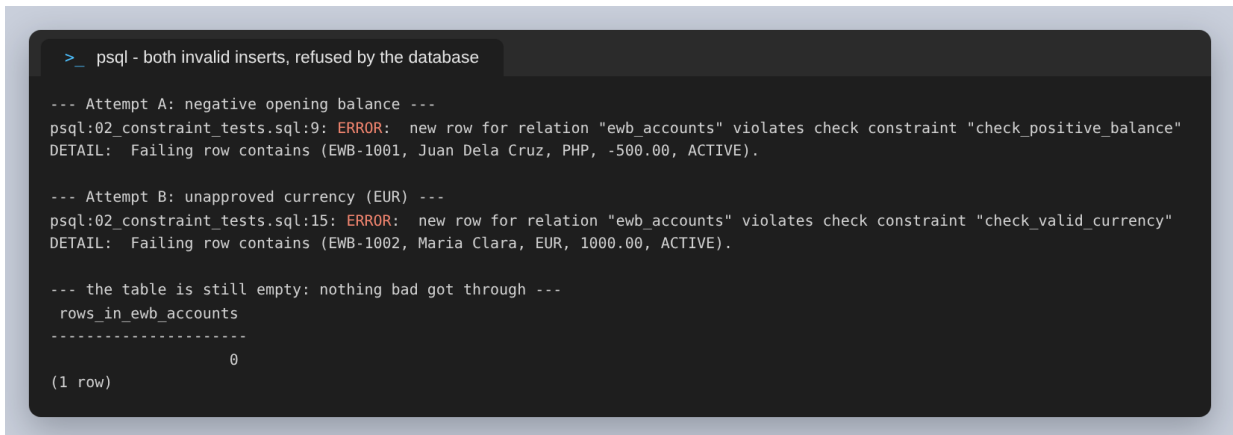


Figure 7: Attempt B: an unapproved currency. Same SQL state, different constraint name — which is how a caller tells the two apart.

SQL state 23514 is `check_violation`. It matters more than the message text: the message is prose and can be reworded between releases, while the five-character SQLSTATE is defined by the standard and is what application code should branch on. An API that parses the words “violates check constraint” out of an error string is one PostgreSQL upgrade away from a silent bug.

Both attempts also demonstrate that the rejection is not partial. The same two statements run through `psql` leave the table exactly as it was:



```
>_ psql - both invalid inserts, refused by the database

--- Attempt A: negative opening balance ---
psql:02_constraint_tests.sql:9: ERROR:  new row for relation "ewb_accounts" violates check constraint "check_positive_balance"
DETAIL:  Failing row contains (EWB-1001, Juan Dela Cruz, PHP, -500.00, ACTIVE).

--- Attempt B: unapproved currency (EUR) ---
psql:02_constraint_tests.sql:15: ERROR:  new row for relation "ewb_accounts" violates check constraint "check_valid_currency"
DETAIL:  Failing row contains (EWB-1002, Maria Clara, EUR, 1000.00, ACTIVE).

--- the table is still empty: nothing bad got through ---
 rows_in_ewb_accounts
-----
                        0
(1 row)
```

Figure 8: Both rejections in one `psql` session, with the row count afterwards. Nothing was written.

Teaching point. The database engine is the last line of defence, and the only one that every writer shares. A front-end API, a batch job and an operator at a `psql` prompt are three different code paths into `ewb_accounts`; the `CHECK` constraint is the only rule all three are guaranteed to pass through. Validating in the API as well is still worth doing — it gives the customer a better message than SQLSTATE 23514 — but it is a convenience, not the guarantee.

pgAdmin shows the same constraints as objects in the tree, which is the view worth knowing about when inheriting a schema nobody documented: the Constraints node under a table lists everything the table enforces without having to query the catalog.

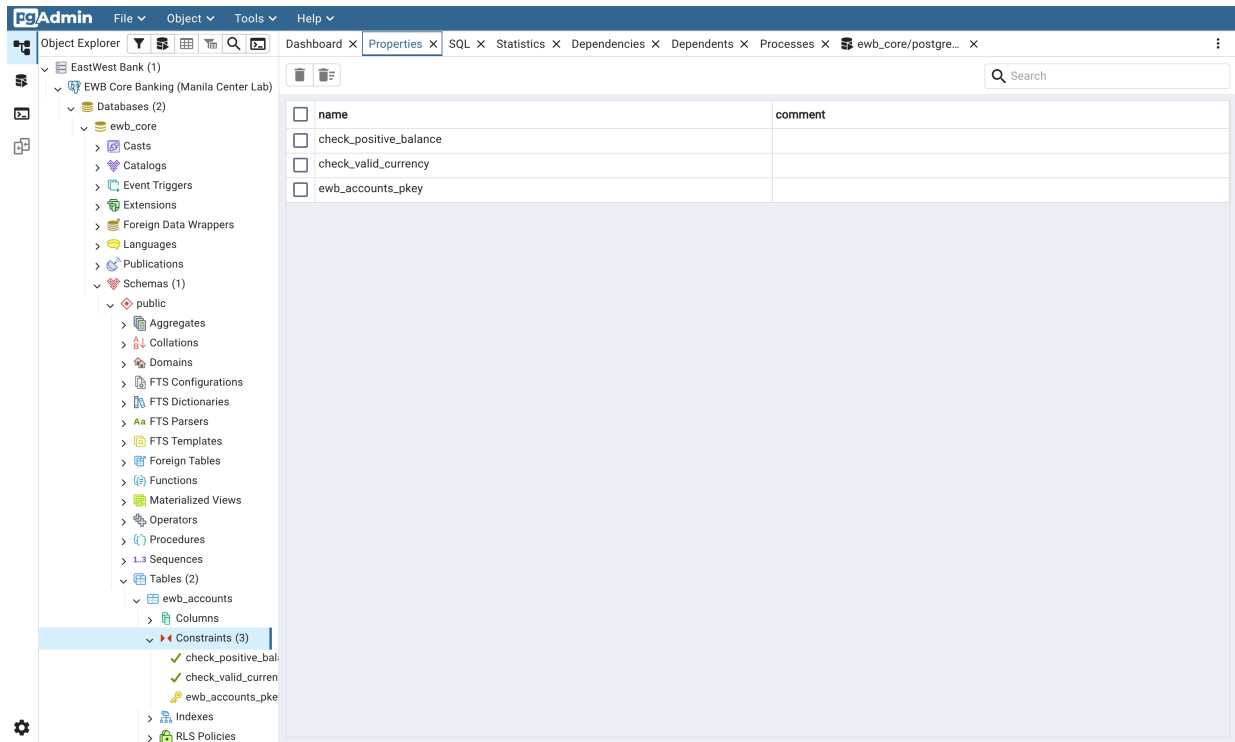


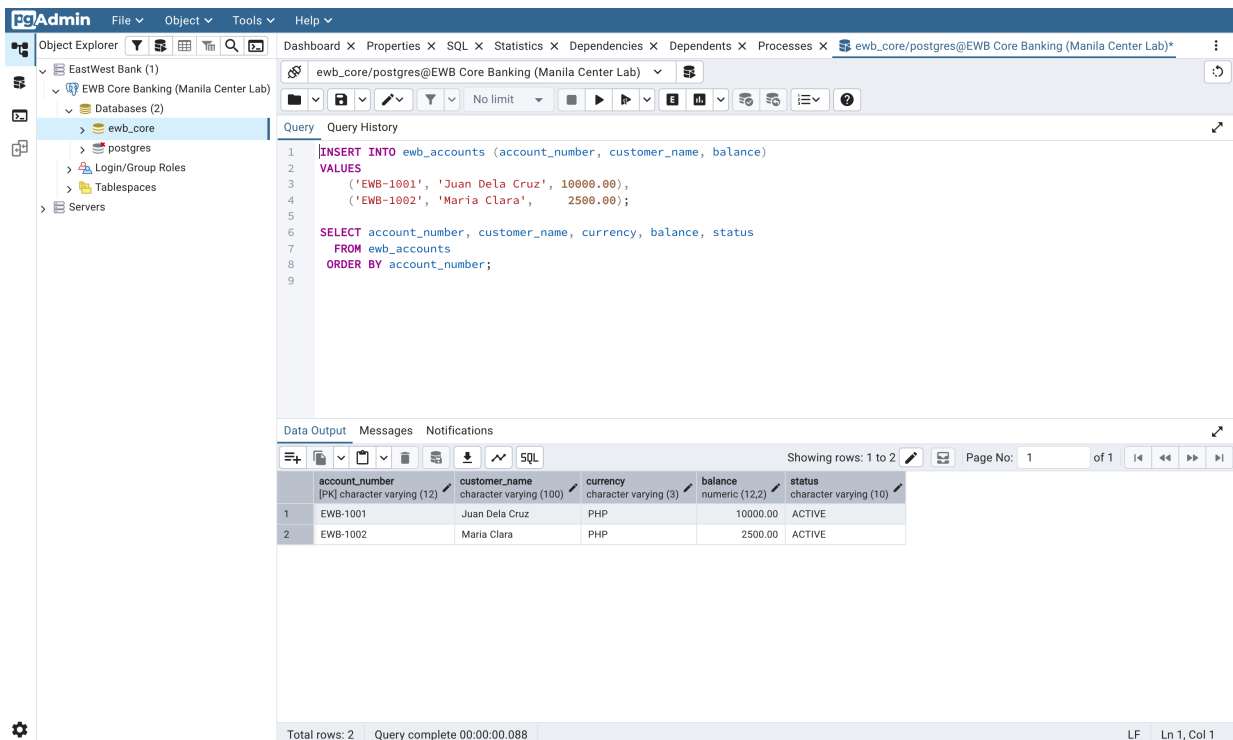
Figure 9: The three constraints on ewb_accounts as pgAdmin sees them: the two CHECKs and the primary key.

4. Exercise 2: Ledger Logging and Relational Queries

A balance on its own is an assertion. A balance with a ledger behind it is an account of how it got that way, and in banking the second is the only acceptable form. This exercise seeds the accounts, then builds the immutable ledger that every later balance change has to be explained by.

Step 2.1: Seeding valid accounts

The same two customers from the failed attempts are inserted again, this time with values the constraints accept: Juan Dela Cruz opens at PHP 10,000.00 and Maria Clara at PHP 2,500.00.



The screenshot shows the pgAdmin interface with a query window open. The query is as follows:

```
1 INSERT INTO ewb_accounts (account_number, customer_name, balance)
2 VALUES
3 ('EWB-1001', 'Juan Dela Cruz', 10000.00),
4 ('EWB-1002', 'Maria Clara', 2500.00);
5
6 SELECT account_number, customer_name, currency, balance, status
7 FROM ewb_accounts
8 ORDER BY account_number;
9
```

The query results are displayed in a table with the following columns: account_number, customer_name, currency, balance, and status. The results show two rows of data:

account_number	customer_name	currency	balance	status
EWB-1001	Juan Dela Cruz	PHP	10000.00	ACTIVE
EWB-1002	Maria Clara	PHP	2500.00	ACTIVE

The status bar at the bottom indicates "Total rows: 2" and "Query complete 00:00:00.088".

Figure 10: Step 2.1: two accounts inserted, then read back. currency and status were never mentioned in the INSERT — the DEFAULTs filled them in.

```
> _ psql - the two valid accounts

INSERT 0 2

--- both accounts, with the columns the DEFAULTs filled in ---
 account_number | customer_name | currency | balance | status
-----+-----+-----+-----+-----
    EWB-1001    | Juan Dela Cruz | PHP      | 10000.00 | ACTIVE
    EWB-1002    | Maria Clara   | PHP      |  2500.00 | ACTIVE
(2 rows)
```

Figure 11: The same insert through psql. INSERT 0 2 is the tag: two rows, and no OID, which is what the 0 means.

The Data Output grid is also showing the declared type under each column heading, which is a detail worth pointing out: balance is numeric(12,2), not a float. Money in a floating-point column is a reconciliation bug waiting for a busy day, because $0.1 + 0.2$ is not 0.3 in binary floating point and a ledger that sums thousands of rows will drift. NUMERIC is exact decimal arithmetic, and it is the only correct choice here.

Step 2.2: The audit ledger, and the foreign key

ewb_transactions is the log. Its account_number REFERENCES ewb_accounts(account_number), which makes an orphaned ledger entry impossible rather than merely unlikely, and it carries two CHECKs of its own: the movement is a DEBIT or a CREDIT and nothing else, and the amount is strictly positive.

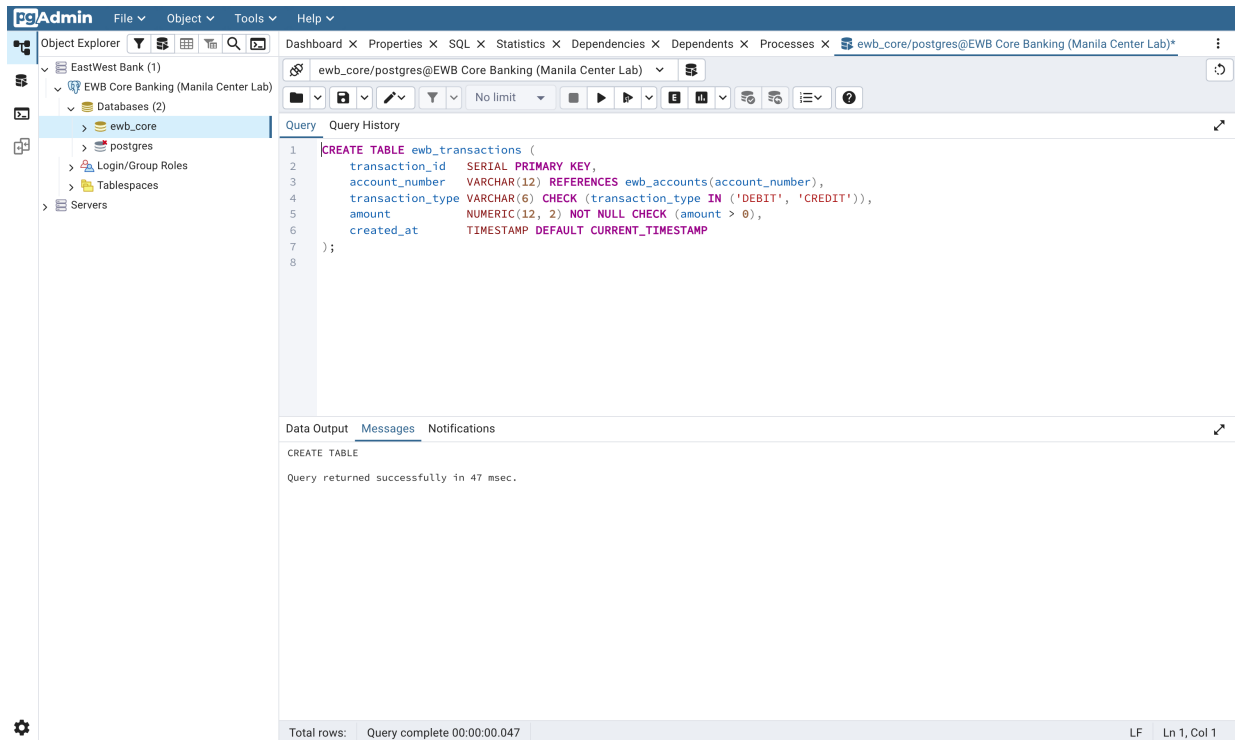


Figure 12: Step 2.2 in pgAdmin. transaction_id is a SERIAL, so the ledger numbers itself.

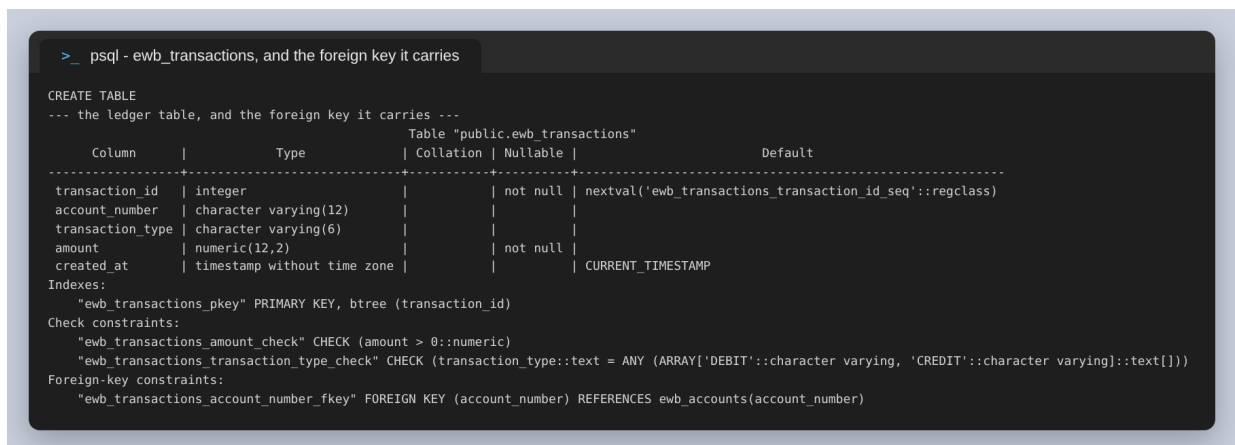


Figure 13: The ledger as stored, including the foreign-key constraint at the bottom.

The amount CHECK is written amount > 0 rather than amount >= 0 on purpose. A zero-value movement is not a transaction; it is either a bug in the caller or an attempt to pad the ledger, and either way the database should not accept it.

All three of those rules were then tested, because a foreign key that has never rejected anything is the same assumption as an untested CHECK:

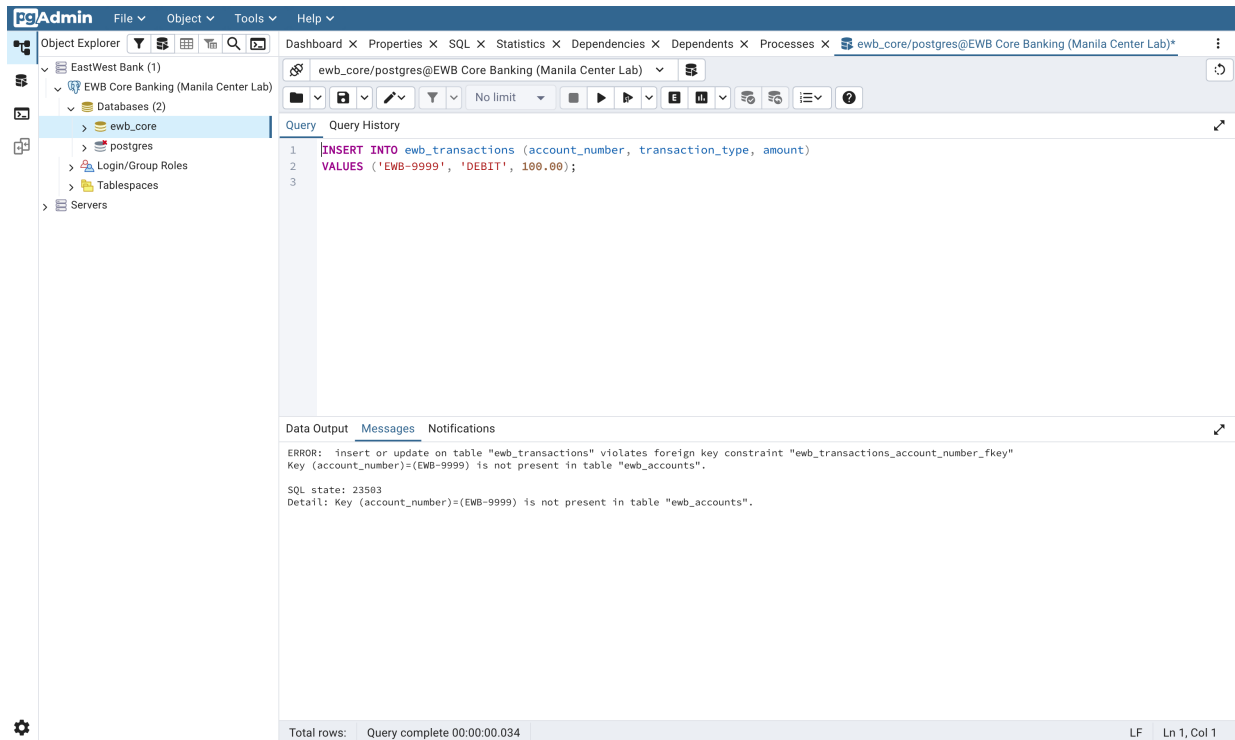


Figure 14: A ledger entry for account EWB-9999, which does not exist. SQL state 23503, foreign_key_violation — a different class of error from the 23514 above.

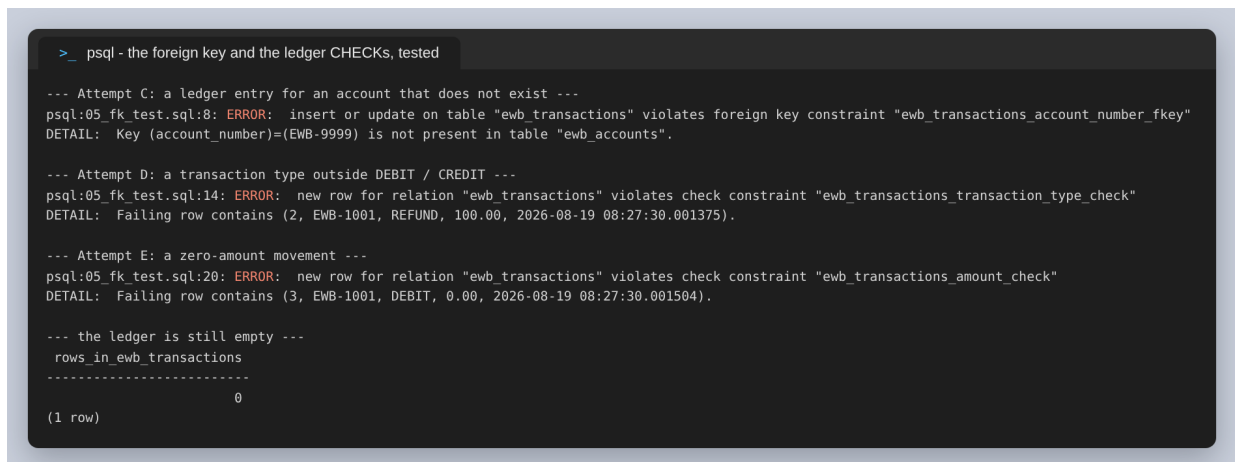


Figure 15: The foreign key, the transaction_type CHECK and the amount CHECK, each refused in turn. The ledger is still empty afterwards.

One thing this step exposed that the lab does not mention. The three rejected inserts consumed sequence values 1, 2 and 3 from the SERIAL, so the first ledger entry that actually commits is transaction_id 4 rather than 1. Sequences are deliberately not transactional — rolling them back would serialise every inserting session behind one another — so gaps in a SERIAL are normal and expected. Any reconciliation process that treats a missing transaction_id as a lost record will raise false alarms; gaps have to be read as “this id was never committed”, not “this row disappeared”.

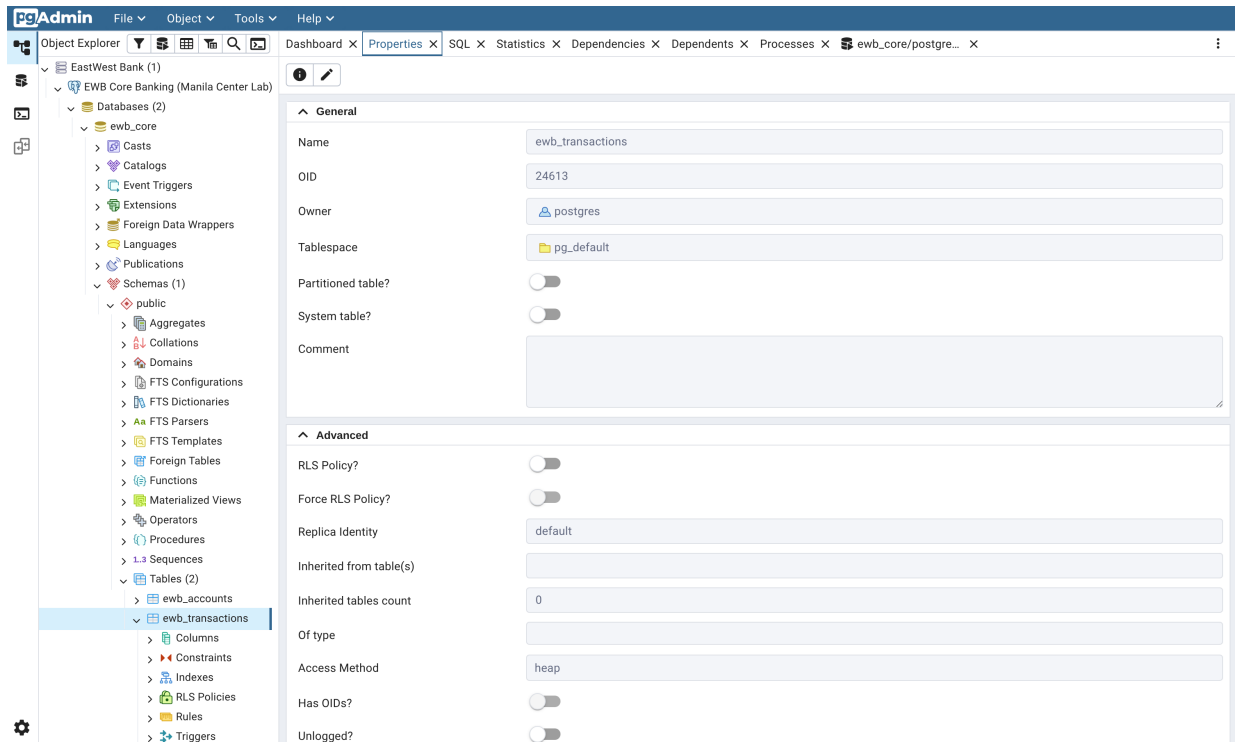


Figure 16: The ledger's constraints in pgAdmin: two CHECKs, the primary key, and the foreign key back to ewb_accounts.

Step 2.3: Filtering and aggregation

With two accounts seeded, the two queries the lab asks for return what they should: one account holds more than PHP 5,000.00, and EWB is holding PHP 12,500.00 in total.

pgAdmin

File Object Tools Help

Object Explorer

- EastWest Bank (1)
 - EWB Core Banking (Manila Center Lab)
 - Databases (2)
 - ewb_core
 - postgres
 - Login/Group Roles
 - Tablespaces

Servers

Dashboard Properties SQL Statistics Dependencies Dependents Processes ewb_core/postgres@EWB Core Banking (Manila Center Lab)*

ewb_core/postgres@EWB Core Banking (Manila Center Lab)

No limit

Query Query History

```
1 SELECT account_number, customer_name, balance
2 FROM ewb_accounts
3 WHERE balance > 5000.00;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	account_number [PK] character varying (12)	customer_name character varying (100)	balance numeric (12,2)
1	EWB-1001	Juan Dela Cruz	10000.00

Total rows: 1 Query complete 00:00:00.045 LF Ln 1, Col 1

Figure 17: Query A in pgAdmin: the accounts above the PHP 5,000.00 threshold.

```
>_ psql - filtering and aggregation
```

```
--- Query A: accounts holding more than PHP 5,000.00 ---
```

account_number	customer_name	balance
EWB-1001	Juan Dela Cruz	10000.00

```
(1 row)
```

```
--- Query B: total deposits EWB is holding across all accounts ---
```

total_ewb_deposits
12500.00

```
(1 row)
```

```
--- Query C: the same total, broken down by currency ---
```

currency	accounts	total_balance
PHP	2	12500.00

```
(1 row)
```

Figure 18: Both queries, plus the same total broken down by currency.

The aggregate is worth naming precisely. `SUM(balance)` over `ewb_accounts` is not EWB's money — it is what EWB owes its customers, which on the bank's own books is a liability. The alias in the query is `total_ewb_deposits` for that reason. It is a habit worth forming early: an aggregate with a vague alias gets read as whatever the reader assumed it meant.

The breakdown by currency returns a single PHP row here, because both seeded accounts are peso accounts. That is the correct result rather than a disappointing one: the `CHECK` permits USD, so the `GROUP BY` is the query that would show a second row the day a dollar account is opened, and summing across currencies without grouping would silently add pesos to dollars.

5. Exercise 3: Transaction Control and Atomic Transfers

A transfer is not one operation. Juan Dela Cruz sending PHP 3,000.00 to Maria Clara through EWB EasyWay is four: debit one balance, credit the other, and write the two ledger entries that account for both movements. If the process dies between the first and the second, the money has left one account without arriving in the other, and the bank is out of balance. Atomicity — the A in ACID — is the property that makes that outcome impossible.

Step 3.1: A successful transfer, BEGIN to COMMIT

All four statements go inside one transaction. Nothing another session can see changes until COMMIT, and if any statement fails, none of them took effect.

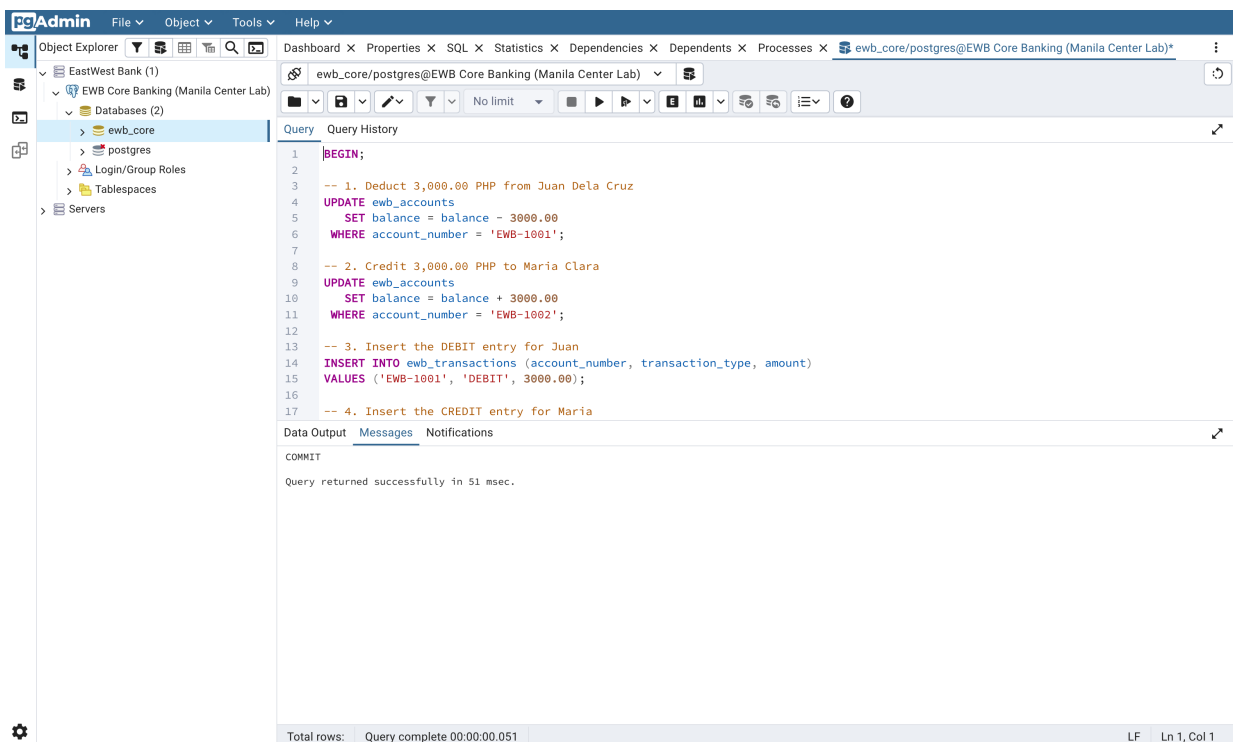


Figure 19: The transfer in pgAdmin: BEGIN at line 1, and the Messages pane reporting COMMIT once all four statements have run.

psql reports a tag per statement, which is a useful way to watch the transaction advance: BEGIN, then UPDATE 1 twice, then INSERT 0 1 twice, then COMMIT. The UPDATE 1 matters — it says exactly one row was matched. An UPDATE 0 would mean the account number was wrong and the money went nowhere, and an UPDATE 2 would mean something far worse about the primary key.


```
> _ psql - the transfer, one statement at a time
```

```
BEGIN;  
BEGIN  
-- 1. Deduct 3,000.00 PHP from Juan Dela Cruz  
UPDATE ewb_accounts  
    SET balance = balance - 3000.00  
    WHERE account_number = 'EWB-1001';  
UPDATE 1  
-- 2. Credit 3,000.00 PHP to Maria Clara  
UPDATE ewb_accounts  
    SET balance = balance + 3000.00  
    WHERE account_number = 'EWB-1002';  
UPDATE 1  
-- 3. Insert the DEBIT entry for Juan  
INSERT INTO ewb_transactions (account_number, transaction_type, amount)  
VALUES ('EWB-1001', 'DEBIT', 3000.00);  
INSERT 0 1  
-- 4. Insert the CREDIT entry for Maria  
INSERT INTO ewb_transactions (account_number, transaction_type, amount)  
VALUES ('EWB-1002', 'CREDIT', 3000.00);  
INSERT 0 1  
-- Commit all four operations permanently  
COMMIT;  
COMMIT
```

Figure 20: The same transfer through psql, one statement and one tag at a time.

The balances afterwards are the ones the lab predicts: Juan at PHP 7,000.00 and Maria at PHP 5,500.00.

pgAdmin

Object Explorer | File | Object | Tools | Help

Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Processes | ewb_core/postgres@EWB Core Banking (Manila Center Lab)*

ewb_core/postgres@EWB Core Banking (Manila Center Lab)

Query | Query History

```
1 SELECT * FROM ewb_accounts ORDER BY account_number;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	account_number [PK] character varying (12)	customer_name character varying (100)	currency character varying (3)	balance numeric (12,2)	status character varying (10)
1	EWB-1001	Juan Dela Cruz	PHP	7000.00	ACTIVE
2	EWB-1002	Maria Clara	PHP	5500.00	ACTIVE

Total rows: 2 | Query complete 00:00:00.045 | LF | Ln 1, Col 1

Figure 21: After the commit: PHP 7,000.00 and PHP 5,500.00.

pgAdmin

Object Explorer | File | Object | Tools | Help

Dashboard | Properties | SQL | Statistics | Dependencies | Dependents | Processes | ewb_core/postgres@EWB Core Banking (Manila Center Lab)*

ewb_core/postgres@EWB Core Banking (Manila Center Lab)

Query | Query History

```
1 SELECT transaction_id, account_number, transaction_type,
2 amount, created_at
3 FROM ewb_transactions
4 ORDER BY transaction_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1 of 1

	transaction_id [PK] integer	account_number character varying (12)	transaction_type character varying (6)	amount numeric (12,2)	created_at timestamp without time zone
1	2	EWB-1001	DEBIT	3000.00	2026-08-19 08:33:48.21847
2	3	EWB-1002	CREDIT	3000.00	2026-08-19 08:33:48.21847

Total rows: 2 | Query complete 00:00:00.050 | LF | Ln 1, Col 1

Figure 22: The two ledger entries the transfer wrote, numbered 4 and 5, sharing one timestamp because CURRENT_TIMESTAMP is fixed for the whole transaction.

Both ledger rows carry the same `created_at`, to the microsecond. That is not a coincidence and it is not a rounding artefact: `CURRENT_TIMESTAMP` in PostgreSQL is the start time of the transaction, not of the statement, so every row written inside one transaction is stamped identically. It is the right behaviour for a ledger — the two halves of a double entry should not appear to have happened at different times — but code that expects `created_at` to order statements within a transaction will not get what it expects. `statement_timestamp()` is the function for that.

Two properties of the result are worth checking rather than assuming, and both are in the verification below. The debits and the credits sum to the same figure, so the ledger nets to zero; and the total deposits are still PHP 12,500.00, unchanged, because an internal transfer moves money between EWB's liabilities without changing what EWB owes in total. If either number had moved, the transfer would have created or destroyed money.

```
> _ psql - balances and ledger after COMMIT

--- balances after the commit ---
account_number | customer_name | balance
-----+-----+-----
EWB-1001       | Juan Dela Cruz | 7000.00
EWB-1002       | Maria Clara   | 5500.00
(2 rows)

--- the ledger the transfer wrote ---
transaction_id | account_number | transaction_type | amount |      created_at
-----+-----+-----+-----+-----
              4 | EWB-1001      | DEBIT            | 3000.00 | 2026-08-19 08:27:30.02034
              5 | EWB-1002      | CREDIT           | 3000.00 | 2026-08-19 08:27:30.02034
(2 rows)

--- double entry check: DEBIT total must equal CREDIT total ---
total_debits | total_credits | net
-----+-----+-----
      3000.00 |      3000.00 | 0.00
(1 row)

--- total deposits held: unchanged by an internal transfer ---
total_ewb_deposits
-----
      12500.00
(1 row)
```

Figure 23: Balances, the ledger, the double-entry check and the unchanged deposit total.

Step 3.2: An overdrawn transfer, and ROLLBACK

Maria Clara now holds PHP 5,500.00 and attempts to send PHP 20,000.00. `check_positive_balance` rejects the `UPDATE`, and the interesting part is what happens next: the transaction is not merely carrying a failed statement, it is aborted. PostgreSQL refuses everything sent after the error until the block is ended.

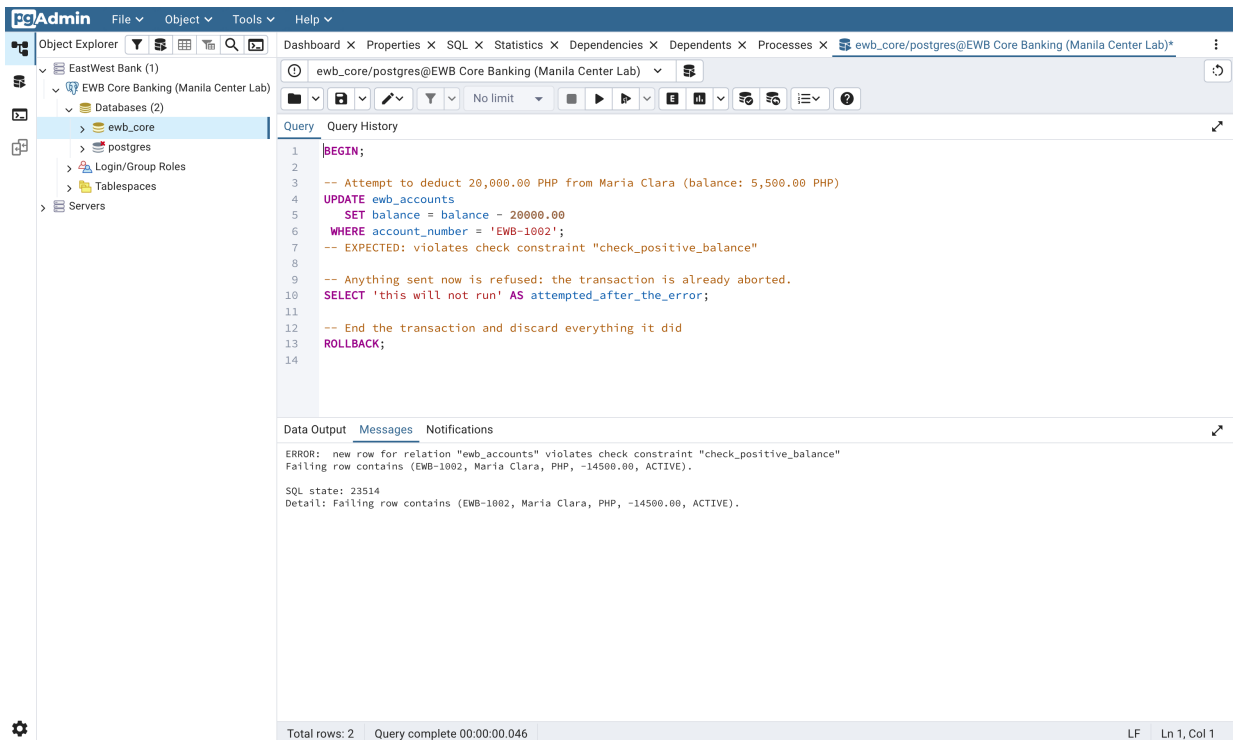


Figure 24: The overdrawn attempt in pgAdmin. The constraint rejects the `UPDATE`, and the `SELECT` on line 10 never runs.

The psql transcript shows the second error explicitly, which is the one freshers should recognise: “current transaction is aborted, commands ignored until end of transaction block”. This is a common cause of confusion in shared psql sessions — one mistake early in a transaction makes every later statement fail with a message that says nothing about what is actually wrong with it.

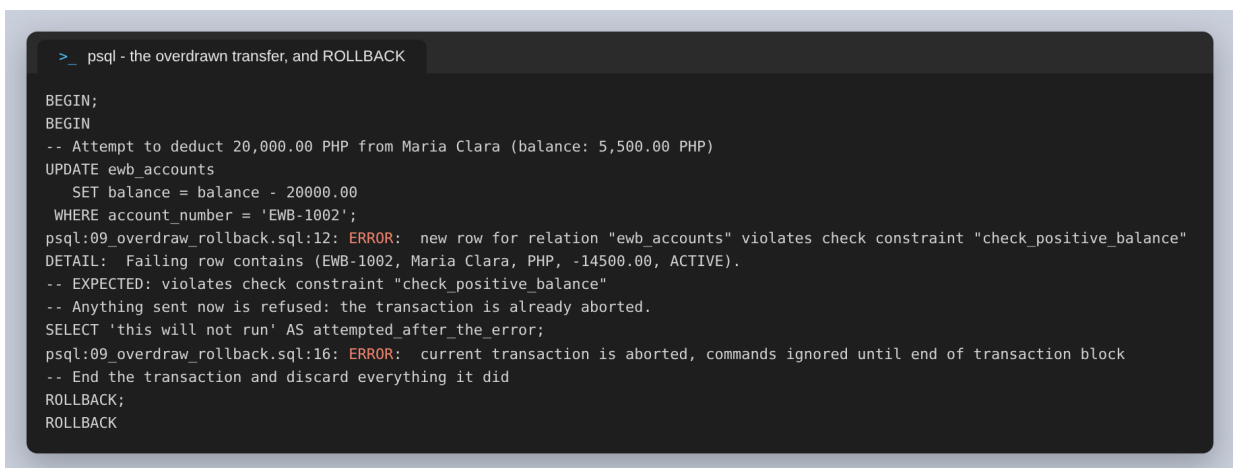


Figure 25: The same block through psql: the constraint violation, then the aborted-transaction error, then `ROLLBACK`.

ROLLBACK ends the block and discards everything in it. Maria's balance is PHP 5,500.00, exactly what it was before the attempt, the ledger still holds two entries and not three, and the deposit total is untouched.

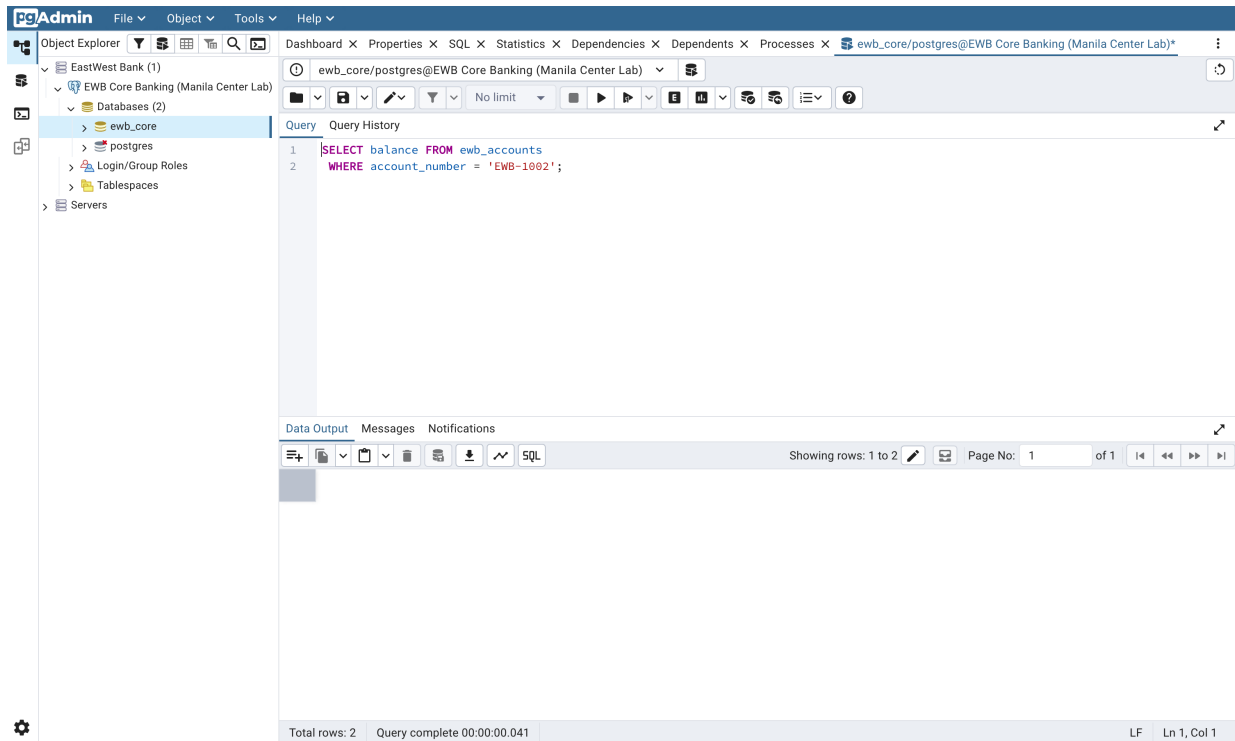


Figure 26: Maria Clara's balance after the rejected transfer: PHP 5,500.00, unchanged.

```

> _ psql - nothing partial survived the rollback

--- Maria Clara, after the rejected transfer ---
account_number | customer_name | balance
-----+-----+-----
EWB-1002      | Maria Clara   | 5500.00
(1 row)

--- both accounts, and the ledger row count: still 2 entries ---
total_ewb_deposits | ledger_entries
-----+-----
          12500.00 |                2
(1 row)

--- customer ledger, joined back to the account master ---
account_number | customer_name | transaction_type | amount | balance_now
-----+-----+-----+-----+-----
EWB-1001      | Juan Dela Cruz | DEBIT           | 3000.00 | 7000.00
EWB-1002      | Maria Clara    | CREDIT          | 3000.00 | 5500.00
(2 rows)

```

Figure 27: The balance, the totals, and the customer ledger joined back to the account master. Two entries, not three.

Critical takeaway. ROLLBACK guarantees zero partial updates. There is no state in which Maria has been debited but the ledger entry is missing, or in which the ledger records a movement that the balance does not reflect. In this case COMMIT would have had the same effect as ROLLBACK, because PostgreSQL will not commit an aborted transaction — it rolls it back and tells you so. That is worth knowing, but it is not a reason to write COMMIT and hope: a transaction that failed should be rolled back explicitly, so the intent is visible in the code.

6. Quick Reference

SQL command	What it does for the banking engine
CHECK (balance >= 0)	A rule the database enforces on every writer, so no application path can post an unapproved overdraft
CONSTRAINT name CHECK (...)	Names the rule, so the error message identifies which rule was broken instead of reporting a generated name
REFERENCES parent(key)	Foreign key: a ledger entry cannot exist for an account that does not
NUMERIC(12, 2)	Exact decimal money. A float column here drifts once a ledger is summed over enough rows
BEGIN	Starts an explicit transaction. Nothing another session can see changes until it ends
COMMIT	Makes every statement in the block durable, together
ROLLBACK	Discards every statement in the block, together. Also the only way out of an aborted transaction
SQLSTATE 23514 / 23503	check_violation and foreign_key_violation. Branch on these, not on the message text

7. Findings

Every step in the lab produced the result it predicts, in both psql and pgAdmin: the two invalid accounts were refused, the transfer left Juan at PHP 7,000.00 and Maria at PHP 5,500.00, and the overdrawn attempt left Maria at PHP 5,500.00 with the ledger unchanged. The findings worth carrying out of it are the ones the exercises imply rather than state:

- Name every constraint. The name is what appears in the error, and it is the difference between an error a caller can act on and one it can only log.
- Branch on SQLSTATE, never on the message. 23514 and 23503 are stable; “violates check constraint” is prose.
- Sequence gaps are normal. Three refused inserts consumed three transaction_ids, so the first committed entry is 4. Reconciliation that treats a gap as a lost row will report failures that never happened.
- CURRENT_TIMESTAMP is per transaction, not per statement. Correct for a double entry; wrong for anything trying to order statements within a transaction.
- Check the invariant, not just the balances. The debits equalling the credits and the deposit total staying put are the two numbers that would have caught a transfer which created money.
- The database is the only rule every writer shares. Validate in the application too, for the sake of the error message, but never instead.

Two limits of this lab are worth stating plainly, because a fresher could otherwise read it as a finished design. First, the balance and the ledger are kept in step by the transaction and by nothing else — there is no constraint that stops a future UPDATE from moving a balance without writing a ledger row. In a real core banking schema that link is enforced, usually by making the balance derived from the ledger or by permitting balance changes only through a stored procedure that writes both. Second, nothing here concerns concurrency: two simultaneous transfers out of the same account were not tested, and the CHECK constraint alone does not prevent the classic case where both read a sufficient balance and only the second one's write fails. Both are the natural next exercises.

A note on presentation: the shell prompts in the psql figures are shown as PowerShell to match the earlier assignments in this series, and the psql output is unedited apart from shortening the absolute paths of the scripts to their file names, which setup.sh does to its own captures so the shortening is applied consistently rather than by hand.